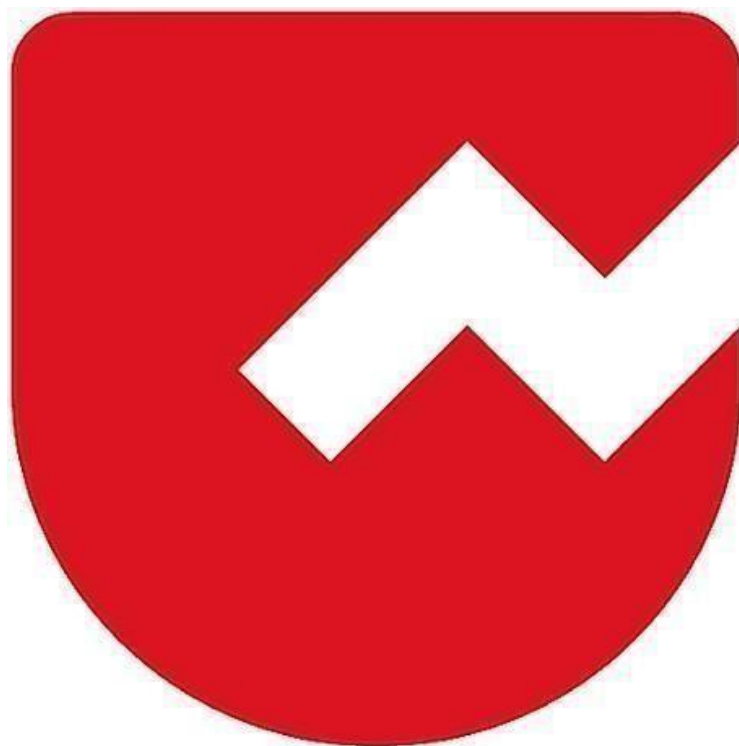


Zoho Schools for Graduate Studies

Notes



Computation of Running Time

Time complexity (Big-O) — quick recap

Algorithm	Time Complexity
For loop	$O(n)$
Nested loops	$O(n^2)$
Binary search	$O(\log n)$
Merge sort / quicksort avg	$O(n \log n)$
Traversing array once	$O(n)$
HashMap get/put (avg)	$O(1)$

Time Complexity

- **For loop – $O(n)$**

If you go through items one by one, the time grows directly with the number of items.

- **Nested loops – $O(n^2)$**

A loop inside another loop. For every item, you again go through all items, so work grows much faster (square of n).

- **Binary search – $O(\log n)$**

Each step cuts the list in half (like guessing a number game by halving the range). Very fast even for big lists.

- **Merge sort / Quicksort average – $O(n \log n)$**

You both divide the data ($\log n$ part) and process all elements (n part). Faster than n^2 , slower than n .

- **Traversing array once – $O(n)$**

You look at each element only one time from start to end.

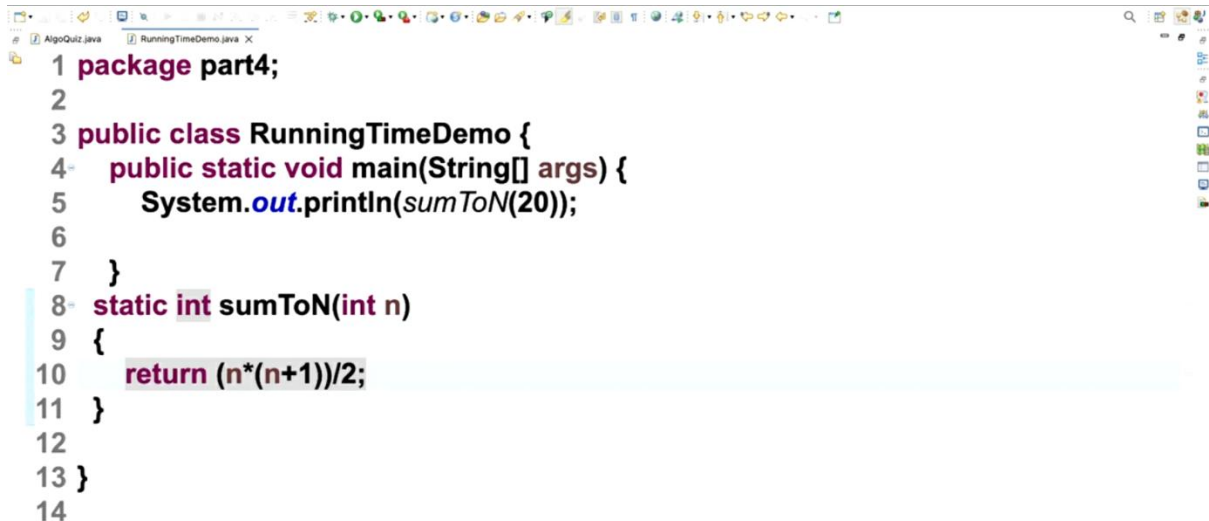
- **HashMap get/put (average) – $O(1)$**

You can directly jump to the needed item without searching through others. Time is constant, doesn't depend on size (on average).

Counting number of operations

1. only constant amount of work $O(1)$

(i)



```

1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(sumToN(20));
6     }
7 }
8 static int sumToN(int n)
9 {
10     return (n*(n+1))/2;
11 }
12
13 }
14

```

Expression:

$$(n * (n + 1)) / 2$$

Count basic operations:

1. $n + 1 \rightarrow$ **1 operation**
2. $n * (\text{result}) \rightarrow$ **1 operation**
3. division by 2 $/ 2 \rightarrow$ **1 operation**
4. final return (sometimes counted as constant) $\rightarrow$ **1 operation**

Total operations

= 3 arithmetic operations
 + 1 return
 = 4 operations (constant number)

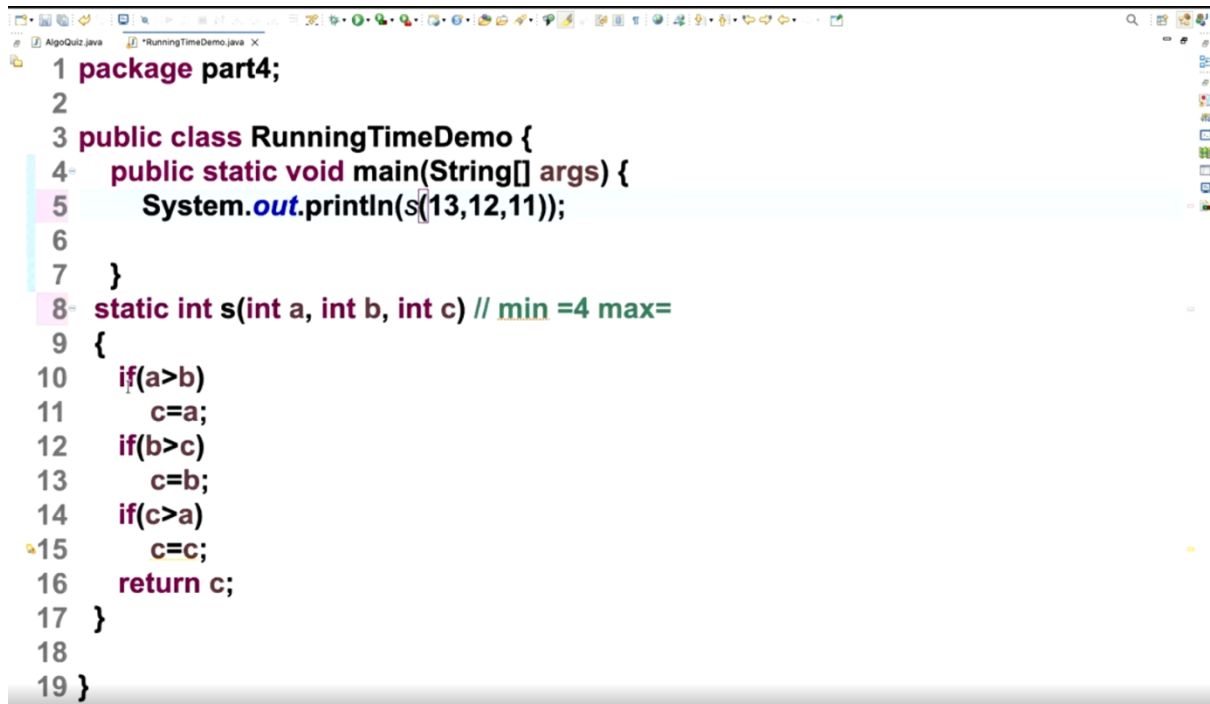
Even if you call it for $n = 10, 1000, 10^6, 10^9$, operations are **still 4** — it doesn't change.

Time Complexity

Since the number of operations **does not depend on n**:

Time Complexity = $O(1)$ *//(constant time)*

(ii)



```
1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(s(13,12,11));
6     }
7 }
8 static int s(int a, int b, int c) // min =4 max=
9 {
10     if(a>b)
11         c=a;
12     if(b>c)
13         c=b;
14     if(c>a)
15         c=c;
16     return c;
17 }
18
19 }
```

We count:

- each **comparison** as 1 operation
- each **assignment** as 1 operation
- return as 1 operation

Expression:

a = 13, b = 12, c = 11

Step-by-step:

1. $a > b \rightarrow 13 > 12 \rightarrow \mathbf{true} \rightarrow$ assignment executes

comparison = 1

assignment $c = a = 1$

Now $c = 13$

2. $b > c \rightarrow 12 > 13 \rightarrow \mathbf{false}$

comparison = 1

3. $c > a \rightarrow 13 > 13 \rightarrow \mathbf{false}$

comparison = 1

4. return c

return = 1

Total operations executed

comparisons: 3

assignments: 1

return: 1

TOTAL = 5 operations

This is **neither** minimum (4) nor maximum (7);

(iii)

```

1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(s(10,12,1));
6     }
7 }
8 static int s(int a, int b, int c) // min =4 max=
9 {
10     if(a>b)
11         c=a;//c=13
12     if(b>c)
13         c=b;
14     if(c>a)
15         c=c;
16     return c;
17 }
18
19 }

```

Expression:

a = 10, b = 12, c = 1

Step-by-step:

1. $a > b \rightarrow 10 > 12 \rightarrow \text{false}$
comparison = 1

2. $b > c \rightarrow 12 > 1 \rightarrow \text{true} \rightarrow$ assignment executes
comparison = 1
assignment $c = b = 1$
Now c = 12

3. $c > a \rightarrow 12 > 10 \rightarrow \text{true} \rightarrow$ assignment executes
comparison = 1
assignment $c = c = 1$

4. return c
return = 1

Total operations executed

comparisons: 3
assignments: 2
return: 1
TOTAL = 6 operations

This is close to max but not full 7 (because first if failed).

Summary Table

Input	Comparisons	Assignments	Return	Total
s(13,12,11)	3	1	1	5
s(10,12,1)	3	2	1	6

MINIMUM OPERATIONS


```

1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(s(10,10,10));
6     }
7 }
8 static int s(int a, int b, int c)
9 {
10     if(a>b)
11         c=a;
12     if(b>c)
13         c=b;
14     if(c>a)
15         c=c;
16     return c;
17 }
18
19 }

```

Expression:

a = 10, b = 10, c = 10

Step-by-step:

- 1 First if ($a > b$)
 - Check: $10 > 10$
 - This is false
 - So the statement $c = a$ does not execute

- 2 Second if ($b > c$)
 - Check: $10 > 10$
 - This is false
 - So the statement $c = b$ does not execute

- 3 Third if ($c > a$)
 - Check: $10 > 10$

- This is false
 - So the statement $c = c$ does not execute
-

4 return c

- c is still 10
 - So the function returns 10
-

Operation counting

- 3 comparisons $\rightarrow a > b, b > c, c > a$
- 0 assignments (because all conditions are false)
- 1 return statement

Total operations executed

$3 + 0 + 1 = 4$ operations

This is the **minimum possible operations** for this program.

Time Complexity = $O(1)$ //(constant time)

2. linear $O(n)$

```
1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(sum(100));
6     }
7 }
8
9 static int sum(int n)
10 {
11     int s=0;
12     for(int i=0;i<n;i++)
13         s=s+i;
14     return s;
15 }
16 }
17
```

We will find:

- ☒ number of operations
- ☒ minimum & maximum (here both same)
- ☒ time complexity
- ☒ short explanation

Step-by-step operation counting

1) Initialization

int s = 0; → 1 operation

int i = 0; → 1 operation

2) Loop condition check

i < n happens:

- **$n + 1$ times**
(because one extra false check when loop ends)

→ **$n + 1$ operations**

3) Increment

$i++$ → n operations

4) Inside loop

$s = s + i$
= addition → n operations
+ assignment → n operations
So inside loop = **$2n$ operations**

5) Return

return s ; → 1 operation

Operation counting

- 3 comparisons → $a > b$, $b > c$, $c > a$
- 0 assignments (because all conditions are false)
- 1 return statement

Total operations executed

Add everything:

1 ($s = 0$)
+ 1 ($i = 0$)

+ (n + 1) comparisons

+ n increments

+ 2n inside loop

+ 1 return

= 1 + 1 + n + 1 + n + 2n + 1

= 4n + 4 operations

So exact operation count = $4n + 4$.

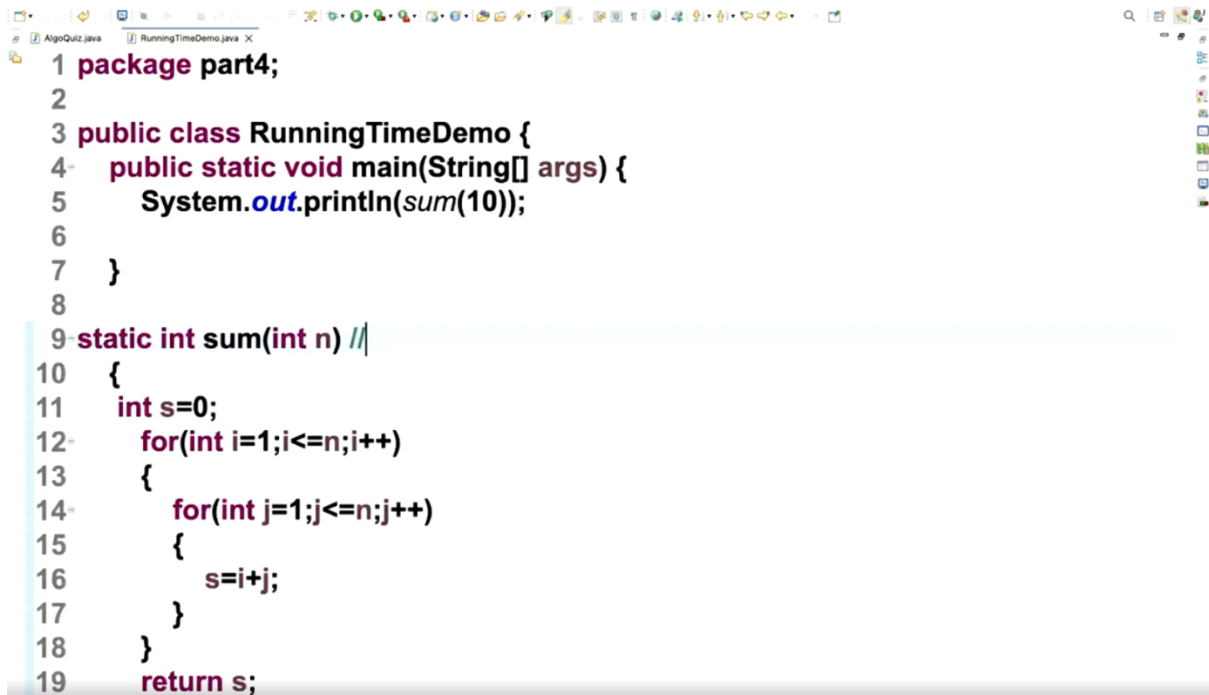
For n = 100

$4(100) + 4 = 404$ operations

Time Complexity = $O(n)$

Since operations grow **proportionally to n**:

$3.O(n^2)$



```

1 package part4;
2
3 public class RunningTimeDemo {
4     public static void main(String[] args) {
5         System.out.println(sum(10));
6     }
7 }
8
9 static int sum(int n) {
10     {
11         int s=0;
12         for(int i=1;i<=n;i++)
13         {
14             for(int j=1;j<=n;j++)
15             {
16                 s=i+j;
17             }
18         }
19         return s;

```

Operation counting like your style

◇ Step 1 — constant statements

int s = 0; // 1 operation

return s; // 1 operation

So far:

= 2 operations

◇ Step 2 — outer loop

for(int i = 1; i <= n; i++)

Operations inside loop header:

- initialization → **i = 1** → 1
- comparison **i <= n** → happens **n + 1** times
- increment **i++** → happens **n** times

Total outer loop control:

$$1 + (n + 1) + n = 2n + 2$$

◇ **Step 3 — inner loop**

for(int j = 1; j <= n; j++)

For **each i**, inner loop runs **n times**

Inner loop control operations:

- initialization $j = 1 \rightarrow 1$
- comparison $j \leq n \rightarrow (n + 1)$ times
- increment $j++ \rightarrow n$ times

So for one outer loop run:

$$1 + (n + 1) + n = 2n + 2$$

But outer loop itself runs **n times**, so multiply:

$$n(2n + 2) = 2n^2 + 2n$$

◇ **Step 4 — statement inside inner loop**

$s = i + j;$

Includes:

- addition $\rightarrow 1$
- assignment $\rightarrow 1$

So:

2 operations per iteration

Number of executions:

$$n \times n = n^2$$

So total:

$2n^2$ operations

Now add everything

outer loop control: $2n + 2$
inner loop control: $2n^2 + 2n$
inner statement: $2n^2$
constants: 2

TOTAL = $4n^2 + 4n + 4$ operations

Time Complexity

Since the number of operations grows proportional to n^2 (because of the two nested loops), the time complexity is $O(n^2)$.

Time Complexity = $O(n^2)$